



Manus → Google Gemini

安全・完全移行 手順書

初心者向け／画面イメージ付き／バックアップ・セキュリティ重視

安全な完全移行の全体像



最重要：秘密鍵・パスワード・本番DB情報はAIへ渡さない

Stripe Secret Key / Webhook Secret / .env / DBパスワード / GitHub Token など

作成日：2026年8月9日

象：Manus WebDev で制作した PingPong Master を、Google Gemini で継続制作できる状態へ移す

この手順書の基本方針：『元データを消さない』『秘密情報を AI へ渡さない』『一度に本番公開しない』の3原則を守ります。

1. 最初に結論：移行は可能です

Manus 公式ドキュメントでは、Website Builder の「Code Control」からコードの一部をコピーでき、さらにアプリの全コードベースをダウンロードできると案されています。したがって、現在の PingPong Master のコードを手元へ取り出し、別の開発環境や Git リポジトリへ移して継続開発することは可能です。

Gemini Apps も、ローカルのファイルやコードフォルダ、または GitHub リポジトリを読み込んでコードについて質問できます。PC 版 Gemini Web では GitHub リポジトリのインポートにも対応しています。

ただし『Manus の全コードを、そのまま全部 Gemini へ貼る』のはおすすめしません。まず秘密情報を除去してから渡します。

おすすめする完成形

- Manus : 現在の原本を保管する場所
- 手元の ZIP : 緊急復元用バックアップ
- GitHub (できれば Private) : 今後のコードの正本履管理
- Gemini : コード解析・修正案・新機能制作を行う AI 作業場所

2. 移行前に知っておく安全ルール

アップロード前の「秘密情報」チェック

☒ そのまま渡さない

sk_live_ABC123...

STRIPE_WEBHOOK_SECRET=whsec_...

DATABASE_URL=postgres://user:pass@...

GITHUB_TOKEN=ghp_...

管理者パスワード / 個人情報

☒ 置き換えて渡す

STRIPE_SECRET_KEY=YOUR_SECRET_KEY

STRIPE_WEBHOOK_SECRET=YOUR_WEBHOOK_SECRET

DATABASE_URL=YOUR_DATABASE_URL

GITHUB_TOKEN=YOUR_GITHUB_TOKEN

ダミー値・説明用の仮データ

絶対にそのままアップロードしないもの

- Stripe Secret Key（例：sk_live_... / sk_test_...）
- Stripe Webhook Secret（例：whsec_...）
- .env、.env.local、.env.production 内の秘密値
- データベースのユーザー名・パスワード・本番接続 URL
- GitHub Personal Access Token
- Firebase / Supabase / AWS 等の秘密鍵 Service Role Key
- 管理者パスワード、利用者の個人情報、購入者データ

公開用キーでも、用途を確認できない場合は一度伏せてから渡す方が安全です。

Gemini のデータ保存について

Google 公式の Gemini Apps Privacy Hub では、Gemini へ送信したプロンプト・ファイル・写真・画面共有などが処理対象になると説明されています。Keep Activity がオンの場合、Gemini Apps Activity に保存されます。将来のチャットがサービス改善のレビュー対象になることを抑えたい場合、Keep Activity をオフにするか、一時チャットを利用する方法があります。

Keep Activity をオフにしても、サービス提供や安全対策等のため会話が最長 72 時間保存される場合があります。したがって、設定に関係なく『秘密鍵は渡さない』ことが最重要です。

3. 作業前の準備

STEP 1 作業用フォルダを作る

PCに「PingPongMaster_Migration」という新しいフォルダを作ります。iPhone/iPad だけでも可能な場面はありますが、全コード移行 ZIP 確認秘密情報除去は PC の方が安全で分かりやすいです。

STEP 2 3つの保存場所を用意する

同じフォルダに次の3つを作ります。

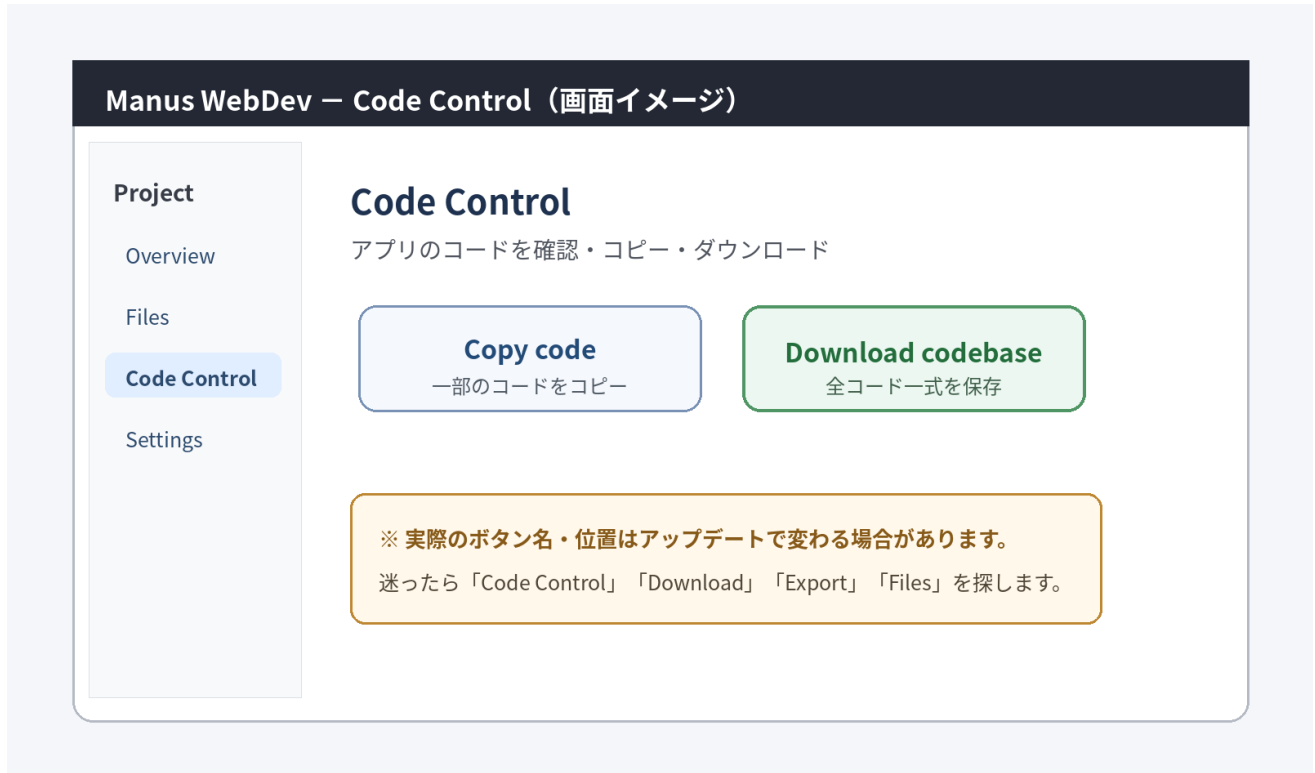
- 01_Manus_Original（触らない原本）
- 02_Safe_Copy（秘密情報を除去する作業用）
- 03_Gemini_Upload（Gemini に渡す最終版）

STEP 3 日付を入れる

バックアップ ZIP の名前を「PingPongMaster_Manus_2026-08-09.zip」のようにします。後でどれが最新版か迷いません。

01_Manus_Original に保存した原本 ZIP は、編集・上書きしないでください。

4. Manus から全コードを取り出す



上の画像は説明用の画面イメージです。Manus のアップデートによりボタン名や位置が変わる場合があります。

STEP 1 PingPong Master プロジェクトを開く

Manus にログインして、現在作成中の PingPong Master を開きます。

STEP 2 Code Control を探す

WebDev 編集面で「Code Control」「Code」「Files」「Project settings」などコード管理に関係する項目を探します。

STEP 3 Download を選ぶ

Manus 公式ドキュメントの Code Control では、全コードベースを Download できると案内されています。ダウンロードできる場合は『コピー』より『全コードダウンロード』を優先します。

STEP 4 原本として保存

ダウンロードした ZIP またはフォルダを「01_Manus_Original」に保存します。

もし Download が見つからない場合

- Code Control 内の「Copy」で重要ファイルを個別コピーする
- Manus のタスク内で VS Code 操作を引きげる場合、Files Explorer から構成を確認する
- むやみにファイルを一つずつ変更せず、まず『どのファイルがあるか』だけ記録する

ポイント不足で AI 編集が止まっても、コード閲覧・ダウンロード機能は使えます。まず Code Control を確認してください。

5. ダウンロードしたコードが揃っているか確認

ダウンロード後に確認するフォルダ構成（例）

pingpong-master/

- └─ src/ または app/
- └─ public/
- └─ components/
- └─ package.json
- └─ README.md
- └─ .env ← 要注意
- └─ .env.local ← 要注意
- └─ node_modules/ ← 通常は移行不要

最低限確認するもの

- ☐ package.json
- ☐ HTML / CSS / JS
- ☐ React / Next.js等のソース
- ☐ 画像・アイコン
- ☐ 設定ファイル
- ☐ データベース構成
- ☐ Stripe連携箇所

プロジェクトごとに構成は違います。上の図と同じでなくても問題ありません。重要なのは『動かすために必要なファイルが一式あるか』です。

確認項目	何を見るか	判定
package.json	React/Next/Vite 等の依存関係	☐
src / app / pages	画面・機能の主要コード	☐
public / assets	画像・アイコン	☐
CSS / styles	レイアウトデザイン	☐
設定ファイル	vite.config / next.config 等	☐
Stripe 関連	Checkout/Payment Link/API 箇所	☐
DB 関連	schema / migrations / client	☐

6. 原本を複製して『安全版』を作る

STEP 1 原本をコピー

01_Manus_Original の内容を丸ごとコピーし、02_Safe_Copy へ入れます。

STEP 2 秘密ファイルを探す

ファイル名で「.env」「secret」「credentials」「key」「token」を検索します。

STEP 3 値をダミー化

秘密値を削除し、YOUR_... のような説明用文字列に置き換えます。

例：

変更前

```
STRIPE_SECRET_KEY=sk_live_XXXXXXXXXXXX
```

変更後

```
STRIPE_SECRET_KEY=YOUR_STRIPE_SECRET_KEY
```

STEP 4 個人情報を確認

利用者名、メールアドレス、住所、注文履歴、管理者アカウントなどがコードや JSON に直接入っていないか確認します。

『秘密値を削除したらサイトが動かなくなる』のは正常です。Gemini へ渡す安全版は、本番秘密値を持たせないためです。後でデプロイ先の環境変数として設定します。

7. Gemini へ渡す方法を選ぶ

方法 A：安全版コードフォルダを直接アップロード（初心者におすすめ）

02_Safe_Copy の内容を確認後、03_Gemini_Upload へコピーします。Gemini の『ファイルを追加』からコードフォルダや対応ファイルをアップロードします。Google 公式ヘルプでは、1つのコードフォルダまたは1つの GitHub リポジトリを、最大 5,000 ファイル・100MB までチャットに追加できると案内されています。

方法 B：Private GitHub へ保存して Gemini からインポート（長期運用におすすめ）

- GitHub で Private リポジトリを作る
- 秘密情報を除去したコードだけをアップロード
- Gemini Web（PC）で『ファイルを追加 → その他のアップロード → コードをインポート』
- GitHub リポジトリまたはブランチ URL を指定

Gemini へ GitHub リポジトリをインポートした後、GitHub 側でコードを更新しても Gemini へ自動同期されません。再読み込み・再インポートが必要です。

8. Gemini へアップロードする

Gemini（画面イメージ）

+ ファイルを追加

コードフォルダ / ZIP / GitHubリポジトリを追加

最初の指示：

「既存機能を削除せず、まず全体構造を解析してください。
秘密情報が含まれていないか確認し、変更前にバックアップ方針を示してください。」

※ GitHubインポートはPC版Gemini Webで利用。モバイルでは新規インポート不可。

STEP 1 Gemini を PC で開く

gemini.google.com へアクセスし、使用する Google アカウントでログインします。

STEP 2 プライバシー設定を確認

個人アカウントでは Keep Activity の設定を確認します。高度な秘密情報は設定に関係なくアップロードしません。

STEP 3 安全版を追加

ローカルファイルから 03_Gemini_Upload を追加するか、Private GitHub をインポートします。

STEP 4 最初は『修正』ではなく『解析』を依

いきなりコードを書き換えさせず、全体構成・使用技術・依存関係・秘密情報残存・不足ファイルを解析させます。

Gemini へ最初に送るプロンプト

このプロジェクトは「PingPong Master」という卓球習サイトです。

Manus から移行した存サイトで、現在のデザインページ機能をできる限り維持したまま開します。

最初はコードを更しないください。

1. フォルダ構成と使用技術を解析

2. 起動に必要な手順を整理
3. 不足ファイルや依存関係を確認
4. .env、API キー、パスワード等の秘密情報が残っていないか確認
5. Stripe・データベース・ログイン機能の接続箇所を一覧化
6. 現在実装済みの機能を一覧化
7. れやすい箇所と移行リスクを報告
8. 変更作業を始める前のバックアップ方針を提示

既存機能を勝手に削除・簡略化・置換しないでください。

9. Gemini での制作を安全に始める

解析が終わったら、1回の依頼でサイト全体を大改造せず、小さな単位で修正します。

安全な修正単位

1. トップページ の 1 セクション
2. ナビゲーション
3. 1つのフォーム
4. 1つの Stripe 購入ボタン
5. 1つの教材ページ
6. 1つの検索機能

毎回 Gemini へ付ける共通ルール

- ・既存機能を削除しない
- ・変更するファイルを先に列挙する
- ・変更理由を説明する
- ・修正前後の差分を提示する
- ・本番キーをコードへ直書きしない
- ・エラー時に戻せるよう 1 更 = 1 コミットを推奨
- ・変更後に確認すべきテスト項目を出す

AI が『全部作り直した方が早い』と提案しても、既存サイトの機能を確認せず全面置換しないことが重要です。

10. ローカルまたはプレビューで動作確認

移行後の「壊れていないか」確認

☐ トップページ 表示・画像・ボタン	☐ ページ移動 戻る / ホーム / メニュー
☐ スマホ表示 文字切れ・横スクロール	☐ Stripe テストモードで購入フロー
☐ ログイン 会員ページへの遷移	☐ 検索 用語・教材が見つかる
☐ 動画 再生・リンク切れ	☐ 法的ページ フッターから開ける

Gemini がコードを修正したら、本番公開前に必ずプレビューまたはローカル環境で確認します。

最低限の通しテスト

- トップページが開く
- 画像・日本語文字が欠けていない
- メニュー・戻る・ホームが動く
- スマホ幅で横にはみ出さない
- 無料コンテンツ／有料コンテンツの別が崩れていない
- 購入ボタンは最初に Stripe テストモードで確認
- 購入後の遷移先が正しい
- ログイン・検索・動画・クイズ等がある場合は一つずつ確認
- 利用規約・プライバシーポリシー・特商法表記へフッターから移動できる

11. Stripe は特に慎重に移行する

Stripe の Secret Key や Webhook Secret は Gemini へ渡さず、デプロイ先の環境変数へ設定します。

移行時の安全な考え方

7. コード側には環境変数名だけ残す
8. 最初は Stripe テストモードで動作確認
9. 商品格成功 URL・キャンセル URL を確認
10. Webhook を使う場合は移行先 URL へ再設定
11. 本番切替後、少額または想定手順で決済フローを再確認

例：コード内では `process.env.STRIPE_SECRET_KEY` のように参照し、秘密値は GitHub や AI チャットに置かず、Hosting 側の Environment Variables で管理します。

12. データベース・会員情報の扱い

Manus のコードを移しても、データベースの『実データ』まで自動的に移るとは限りません。Manus のプロジェクトコピー機能についても、公式ドキュメントではデータベースのスキーマはコピーされる一方、実データは新しいコピーへ転送されないケースが説明されています。

確認すること

- どのDBを使用しているか（Manus 内蔵 / Supabase / Firebase / その他）
- テーブル・スキーマはどこに定義されているか
- ユーザーや購入履歴など本番データのエクスポート方法
- データのバックアップがあるか
- 新しい環境へ移す際の個人情報保護

会員データ・購入者データがある場合、コード移行とは別工程として扱います。データを Gemini へ直接アップロードしないでください。

13. GitHub を『正本』にするおすすめ運用

完全移行後は、Manus や Gemini の中だけに最新版を置かず、GitHub の Private リポジトリをマスターコードにする方法が安全です。

Gemini で修正案 → ローカル/開発環境で確認 → GitHub へ保存 → Hosting へデプロイ

GitHub のメリット

- いつ誰が何を変えたか履歴が残る
- 壊れた場合に前の版へ戻せる
- Manus ポイントがなくてもコード原本は残る
- Gemini・ChatGPT・VS Code 等へ作業環境を変えやすい

14. 失敗しやすい移行方法

避ける	安全な代替
全ファイルを何も確認せず Gemini へ貼る	秘密情報を除去した Safe Copy を作る
原本を直接編集する	原本→作業コピー→Gemini 用の 3 段階保存
本番 Stripe で最初から試す	テストモードで購入導線を確認
AI に全サイトを一括で作り直させる	機能単位で小さく変更する
GitHub へ.env をコミットする	.gitignore で除外し Hosting 側へ設定
移行直後に旧 Manus サイトを削除	新サイトの完全動作確認まで残す

15. 完全移行チェックリスト

バックアップ

- ☐ Manus 全コードを保存した
- ☐ 原本 ZIP を更せず保管した
- ☐ 別の場所にもバックアップした

セキュリティ

- ☐ .env を除外した
- ☐ Stripe 秘密鍵を除外した
- ☐ DB パスワードを除外した
- ☐ 個人情報を除外した

Gemini

- ☐ Safe Copy だけをアップロードした
- ☐ 最初に解析だけを依頼した
- ☐ 既存機能一覧を作った

動作確認

- ☐ トップページ確認
- ☐ スマホ表示確認
- ☐ ナビゲーション確認
- ☐ Stripe テスト確認
- ☐ 法的ページ確認

運用

- ☐ GitHub Private へ保存
- ☐ README に起動方法を記録
- ☐ 環境一を安全に別管理
- ☐ 旧 Manus 版をすぐ削除していない

16. PingPong Master 向け推移行順序

PingPong Master では、教材・決済・会員機能・ランキング・動画など複数機能が絡むため、次の順序をおすすめします。

12. Manus から全コードをダウンロード
13. 原本バックアップを固定
14. 秘密情報を除去
15. Gemini で構造解析
16. トップページを再現
17. ナビゲーションと各ページを確認
18. 法的ページを確認
19. 無料教材を確認
20. 有料教材のアクセス制御を確認
21. Stripe をテストモードで再接続
22. ログイン・会員機能を確認
23. 問題集・用語辞典・検索を確認
24. 動画・AI 機能を確認
25. スマホ/iPad/PC で最終検査
26. 新しい公開 URL へデプロイ
27. 旧 Manus サイトは一定期間バックアップとして残す

最初に『見た目』だけに移すのではなく、最後まで購入・会員・教材へ到達できるかを通して確認することが完全移行の条件です。

17. 困ったときの判断表

状況	やること	やらないこと
Manus Download が見つからない	Code Control/Files/VS Code を確認	無計画に一つずつコピペ
Gemini でエラー	フォルダ容量・ファイル数・依存関係確認	本番秘密鍵を追加して解決しようとする
見た目が崩れた	変更前コミットへ戻す	さらに大規模変更を重ねる
Stripe が動かない	テストモード・環境変数・URL 確認	本番キーをチャットへ貼る
DB が空	別途バックアップ/移行方法を確認	利用者データを AI へ送る

18. 公式情報・参考リンク

Manus Code Control : 全コードのコピー／ダウンロード

<https://manus.im/docs/website-builder/code-control>

Manus Make a Copy : コピー時のコード・Secrets・DB・Stripe の扱い

<https://manus.im/docs/website-builder/make-a-copy>

Manus Help: Browser/VS Code take over : Manus 内の VS Code 操作引継ぎ

<https://help.manus.im/en/articles/11711218-how-can-i-take-over-manus-browser-or-vs-code>

Gemini: ファイルをアップロードして分析 : コードフォルダ・ファイル数・容量

<https://support.google.com/gemini/answer/14903178>

Gemini: GitHub リポジトリをインポート : PC 版での GitHub コードインポート

<https://support.google.com/gemini/answer/16176929?hl=ja>

Gemini Apps Privacy Hub : アップロード内容・データ処理

<https://support.google.com/gemini/answer/13594961>

Gemini Apps Activity : Keep Activity・削除・保存

<https://support.google.com/gemini/answer/13278892>

最後に

この移行で最も大切なのは、Gemini へ『全部コピペすること』ではありません。現在の PingPong Master を壊さず、復元できる状態を確保し、秘密情報を切り離れたうえで、開発場所だけを安全に移すことです。

最初の実作業は『Manus の PingPong Master を開き、Code Control で Download が見えるか確認する』ところから始めてください。